# Sistema Académico y refactor por módulos con Ports

Lab 03 — Ingeniería de Software III

Diego López Camacho — Universidad Industrial de Santander

14 de octubre de 2026

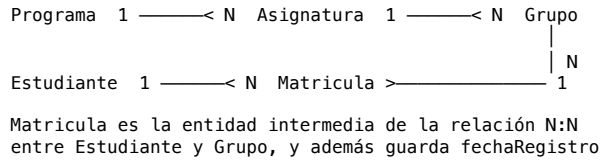
## 1. Qué se construyó

El laboratorio tuvo dos partes con propósitos distintos. La **Parte 1** agregó funcionalidad: el flujo académico `Programa → Asignatura → Grupo → Matrícula` sobre el proyecto `demo-academico` del Lab 02, manteniendo el estilo de la Opción A (DTO + Handler + Service + Controller + `ApiResponse`). La **Parte 2** no agregó ninguna funcionalidad: reorganizó el código en módulos y cambió la forma en que esos módulos se comunican, con una regla arquitectónica explícita que el módulo `matriculas` debía cumplir.

El criterio de éxito de cada parte es diferente, y conviene decirlo desde el principio. En la Parte 1 el éxito es que las reglas de negocio funcionen. En la Parte 2 el éxito es que **no cambie absolutamente nada** desde fuera: mismos endpoints, mismas respuestas, mismos códigos de estado. Un refactor que altera el comportamiento observable no es un refactor, es un cambio funcional disfrazado.

## 2. Parte 1 — Implementación funcional

### 2.1 Modelo del dominio




Se crearon cuatro entidades JPA. `Programa` tiene código único; `Asignatura` referencia a un programa; `Grupo` tiene código único y `cupoMax`, y referencia a una asignatura; `Matricula` referencia a estudiante y grupo con una restricción de unicidad sobre la pareja.

Esa restricción merece un comentario. La regla "un estudiante no se matricula dos veces en el mismo grupo" se verifica en el servicio con `existsByEstudianteIdAndGrupoId`, pero además está declarada en la base de datos:

```
@Table(name = "matricula", uniqueConstraints = {
    @UniqueConstraint(name = "uk_matricula_estudiante_grupo",
                      columnNames = {"estudiante_id", "grupo_id"})
})
```

La validación en el servicio da un mensaje de error legible; la restricción en la base de datos garantiza la invariante incluso si dos peticiones concurrentes pasan la verificación a la vez. Son dos mecanismos para la misma regla con propósitos distintos, y conviene tener los dos. Al arrancar, Hibernate creó las cuatro:

```
add constraint uk_programa_codigo             unique (codigo)
add constraint uk_grupo_codigo                unique (codigo_grupo)
add constraint uk_matricula_estudiante_grupo  unique (estudiante_id, grupo_id)
add constraint UKtbc5806vbo6w5mcbmkqom1m0n    unique (email)
```

### 2.2 Las reglas de matrícula

Las tres reglas quedaron en `MatriculaServiceImpl`, en este orden:

```
// 1) existencia → NotFoundException (404)
Estudiante estudiante = estudianteRepo.findById(estudianteId)
        .orElseThrow(() -> new NotFoundException("Estudiante no encontrado: " + estudianteId));
Grupo grupo = grupoRepo.findById(grupoId)
        .orElseThrow(() -> new NotFoundException("Grupo no encontrado: " + grupoId));

// 2) no duplicado → BusinessException (400)
if (matriculaRepo.existsByEstudianteIdAndGrupoId(estudianteId, grupoId)) {
    throw new BusinessException("El estudiante ya está matriculado en ese grupo");
}

// 3) cupo → BusinessException (400)
long matriculados = matriculaRepo.countByGrupoId(grupoId);
```

```
if (matriculados >= grupo.getCupoMax()) {
    throw new BusinessException("El grupo no tiene cupo disponible");
}
```

El orden no es arbitrario: primero se comprueba que las entidades existan, porque las dos reglas siguientes necesitan datos de ellas; el duplicado va antes que el cupo porque es más barato de verificar y porque es el error más probable del usuario.

### 2.3 Dónde está el acoplamiento que la Parte 2 va a corregir

Esta implementación funciona, pero su lista de dependencias es reveladora:

```
public MatriculaServiceImpl(EstudianteRepository estudianteRepo,
                            GrupoRepository grupoRepo,
                            MatriculaRepository matriculaRepo) { ... }
```

El servicio de matrículas inyecta los repositorios de **otros dos módulos**. Es decir, conoce cómo se persisten entidades que no le pertenecen. Si mañana el acceso a estudiantes cambiara —si pasara a consultarse a un sistema externo de la universidad, por ejemplo—, habría que modificar el código de matrículas, que no tenía por qué enterarse. El enunciado acepta esto en la Parte 1 “para tener la funcionalidad lista”, y es justo lo que la Parte 2 elimina.

### 2.4 Guion de pruebas

Se automatizaron los casos del enunciado en `guion-pruebas.sh`, pensado desde el principio para poder ejecutarse **idéntico** antes y después del refactor. Resultado en la Parte 1:

```
===== 1) POST /api/programas =====
{"success":true,"message":"Programa creado","data":{"id":4,"codigo":"PROG-ING",...}}
HTTP 201

===== 2) POST /api/asignaturas (con programaId) =====
{"success":true,"message":"Asignatura creada","data":{"id":11,"codigo":"ASIG-IS3",
 "creditos":3,"programaId":4,"programaNombre":"Ingeniería de Software"}}
HTTP 201

===== 3) POST /api/grupos (asignaturaId, cupoMax=2) =====
{"success":true,"message":"Grupo creado","data":{"id":21,"codigoGrupo":"G-IS3-01",
 "cupoMax":2,"asignaturaId":11,...}}
HTTP 201

===== 5) matrícula 1 (debe pasar) =====
{"success":true,"message":"Matrícula registrada","data":{"id":1,"estudianteId":51,
 "estudianteNombre":"Estudiante Lab03 1","grupoId":21,"codigoGrupo":"G-IS3-01",...}}
HTTP 201

===== 6) matrícula 2 (debe pasar, llena el cupo) =====
{"success":true,"message":"Matrícula registrada","data":{"id":2,"estudianteId":52,...}}
HTTP 201

===== 7) matrícula 3 (debe FALLAR por cupo) =====
{"success":false,"message":"El grupo no tiene cupo disponible","data":null}
HTTP 400

===== 8) matrícula duplicada (debe FALLAR por duplicado) =====
{"success":false,"message":"El estudiante ya está matriculado en ese grupo","data":null}
HTTP 400

===== 9) estudiante inexistente =====
{"success":false,"message":"Estudiante no encontrado: 999999","data":null}
HTTP 404

===== 10) grupo inexistente =====
{"success":false,"message":"Grupo no encontrado: 999999","data":null}
HTTP 404
```

Las dos matrículas exitosas llenaron el grupo de `cupoMax = 2`, la tercera fue rechazada por cupo y el intento repetido por duplicado. Los errores salen como respuestas controladas con la estructura `ApiResponse`, no como stacktraces.

Conteos tras el seeder y el guion (paso A8):

| Tabla | Registros |
|---|---|
| programa | 4 (3 del seeder + 1 del guion) |
| asignatura | 11 (10 + 1) |
| grupo | 21 (20 + 1) |
| estudiante | 53 (50 + 3) |
| matricula | 2 |

### 2.5 Un error que encontré en el guion

Al correr el guion por primera vez después del refactor, los identificadores no coincidían: el grupo salía con `id` 1 en lugar de 21. Parecía una regresión del refactor, pero no lo era.

La causa es que **Spring abre el puerto HTTP antes de terminar de ejecutar los `CommandLineRunner`**. Mi guion esperaba a que la API respondiera y arrancaba de inmediato, de modo que corría contra una base a medio sembrar. Verificándolo con el conteo en distintos momentos se ve claramente:

```
  (consultando apenas responde la API)   programas: 0   grupos: 20   estudiantes: 39
  (tres segundos después)                programas: 3   grupos: 20   estudiantes: 50
```

Se corrigió añadiendo al guion una espera explícita hasta que los seeders terminen, y con eso los identificadores volvieron a ser reproducibles. La lección es la que ya habíamos visto en Docker con `depends_on`: **que un servicio acepte conexiones no significa que esté listo**. Son dos condiciones distintas y confundirlas produce fallos intermitentes difíciles de diagnosticar.

## 3. Parte 2 — Refactor por módulos con Ports

### 3.1 De paquetes técnicos a módulos de negocio

La estructura del Lab 02 agrupaba por capa técnica: todos los modelos juntos, todos los repositorios juntos, todos los controladores juntos. El refactor la cambió por una agrupación por módulo de negocio:

```
ANTES (por capa técnica)          DESPUÉS (por módulo de negocio)

co.edu.demoacademico              co.edu.demoacademico
├── api                           ├── common
├── exception                     │   ├── api          ApiResponse, ResponseBuilder
├── model                         │   ├── exception    Business, NotFound
│   ├── Estudiante                │   └── controller   GlobalExceptionHandler
│   ├── Programa                  ├── config           los dos seeders
│   ├── Grupo ...                 ├── estudiantes
├── repository                    │   ├── model  repository  service
│   ├── EstudianteRepository      │   ├── port   handler     controller  dto
│   ├── GrupoRepository ...       ├── programas
├── service                       ├── asignaturas
├── handler                       ├── grupos
├── controller                    │   └── port
└── dto                           └── matriculas
```

El cambio de criterio tiene una consecuencia práctica inmediata: con la estructura anterior, tocar el módulo de matrículas significaba abrir seis carpetas distintas; con la nueva, todo lo de matrículas está en un solo sitio. Lo que cambia junto, vive junto.

Se movieron 47 archivos. `common` recoge lo transversal —`ApiResponse`, `ResponseBuilder`, las dos excepciones y el manejador global—, que es lo que el enunciado especifica. Los dos seeders se dejaron en `config` porque son código de arranque de la aplicación y necesitan importar de varios módulos a la vez; no pertenecen a ninguno en particular.

### 3.2 Los Ports

Se crearon dos interfaces, cada una en el módulo **dueño** de la entidad:

```
package co.edu.demoacademico.estudiantes.port;

public interface EstudianteQueryPort {
    Estudiante obtenerPorId(Long id);
}
```

```
package co.edu.demoacademico.grupos.port;

public interface GrupoQueryPort {
    Grupo obtenerPorId(Long id);
}
```

Dos detalles de diseño importan aquí. El primero es **dónde vive el puerto**: en el módulo que posee la entidad, no en el que la consume. El módulo de estudiantes declara qué parte de sí mismo expone; no es el de matrículas quien decide qué quiere. El segundo es **qué tan pequeño es**: una sola operación. Un puerto que expusiera el repositorio completo no habría resuelto nada, porque el acoplamiento habría seguido siendo total; lo que lo convierte en un contrato útil es que solo ofrece lo que el otro módulo realmente necesita.

Los servicios dueños los implementan sin cambiar su código, porque ya tenían ese método:

```
public class EstudianteServiceImpl implements EstudianteService, EstudianteQueryPort { ... }
public class GrupoServiceImpl      implements GrupoService,      GrupoQueryPort      { ... }
```

### 3.3 Matriculas depende solo de contratos

```
@Service
@Transactional
public class MatriculaServiceImpl implements MatriculaService {

    // Solo Ports de otros módulos y el repositorio propio.
    private final EstudianteQueryPort estudiantePort;
    private final GrupoQueryPort grupoPort;
    private final MatriculaRepository matriculaRepo;

    @Override
    public Matricula matricular(Long estudianteId, Long grupoId) {
        Estudiante estudiante = estudiantePort.obtenerPorId(estudianteId);
        Grupo grupo = grupoPort.obtenerPorId(grupoId);
        ...
    }
}
```

La comparación con la versión de la Parte 1 muestra qué cambió exactamente:

| | Parte 1 | Parte 2 |
|---|---|---|
| Dependencias ajenas | `EstudianteRepository`, `GrupoRepository` | `EstudianteQueryPort`, `GrupoQueryPort` |
| Tipo de dependencia | implementación de persistencia | interfaz |
| Quién lanza el `NotFound` | el propio módulo de matrículas | el módulo dueño de la entidad |
| Líneas de la regla | iguales | iguales |

Ese tercer punto es un beneficio que no esperaba y que vale la pena señalar. En la Parte 1, matrículas construía el mensaje `"Estudiante no encontrado: " + id`; ahora lo lanza `EstudianteServiceImpl`. El mensaje de error de una entidad pasa a ser responsabilidad de su módulo, de modo que si cambia la forma de reportar estudiantes no encontrados, se cambia en un solo lugar.

### 3.4 Verificación arquitectónica

El paso B7 pide comprobar la regla por inspección de imports. Se automatizó en `verificar-arquitectura.sh` para que sea repetible en lugar de una revisión manual:

```
$ ./verificar-arquitectura.sh
Regla: matriculas no depende de repositorios ajenos, solo de Ports

  OK     matriculas NO importa estudiantes.repository
  OK     matriculas NO importa grupos.repository
  OK     matriculas SÍ importa EstudianteQueryPort
  OK     matriculas SÍ importa GrupoQueryPort

Resultado: se cumple la regla arquitectónica.
```

Los imports reales de `MatriculaServiceImpl` lo confirman:

```
import co.edu.demoacademico.common.exception.BusinessException;
import co.edu.demoacademico.estudiantes.model.Estudiante;
import co.edu.demoacademico.estudiantes.port.EstudianteQueryPort;
import co.edu.demoacademico.grupos.model.Grupo;
import co.edu.demoacademico.grupos.port.GrupoQueryPort;
import co.edu.demoacademico.matriculas.model.Matricula;
import co.edu.demoacademico.matriculas.repository.MatriculaRepository;
```

De otros módulos solo entran dos cosas: el **modelo** (necesario, porque la matrícula guarda referencias a estudiante y grupo) y el **puerto**. Ningún repositorio ajeno.

Que la verificación sea un script y no una inspección a ojo es deliberado. Una regla arquitectónica que solo vive en la cabeza del equipo se erosiona sola: basta que alguien importe el repositorio directamente para que el aislamiento se rompa, y **nada falla visiblemente**. El código compila, los tests pasan y el acoplamiento vuelve sin que nadie se entere. Un script ejecutable convierte la regla en algo comprobable, y podría conectarse al pipeline de integración continua para que falle el build si se incumple.

### 3.5 No regresión

La prueba de que el refactor no cambió el comportamiento es ejecutar el mismo guion y comparar las salidas. Normalizando los campos de tiempo, que obviamente cambian entre ejecuciones:

```
$ diff <(parte1 normalizada) <(parte2 normalizada)
IDÉNTICO — no hay regresión (56 líneas comparadas)
```

No solo coinciden los códigos de estado y los mensajes: coinciden los **identificadores generados**. El programa sigue siendo el 4, la asignatura la 11, el grupo el 21 y los estudiantes el 51, 52 y 53. Los endpoints, los cuerpos de respuesta y la estructura `ApiResponse` son los mismos byte a byte.

Esto es exactamente lo que debe pasar: se movieron 47 archivos, cambiaron todos los paquetes y se invirtió la dirección de dos dependencias, y desde fuera la aplicación es indistinguible. La prueba de un buen refactor es que nadie que use el sistema pueda notar que ocurrió.

## 4. Entregable y checklist

| Entregable | Dónde está |
|---|---|
| Proyecto con Parte 1 completa | commit `feat(lab03): Parte 1` |
| Proyecto refactorizado con Ports | commit `refactor(lab03): Parte 2` |
| 2 matrículas exitosas | `evidencia-parte1.txt`, casos 5 y 6 |
| 1 error por cupo | `evidencia-parte1.txt`, caso 7 |
| 1 error por duplicado | `evidencia-parte1.txt`, caso 8 |
| Mismo guion tras el refactor | `evidencia-parte2.txt` + diff idéntico |
| README con ejecución, URLs y Ports | `README.md` |

**Parte 1**

- Entidades `Programa`, `Asignatura`, `Grupo`, `Matricula` creadas.
- Reglas de matrícula: existencia (404), duplicado (400) y cupo (400).
- DTO + Handler + Service + Controller con `ApiResponse` en los cuatro módulos.
- Seeder de programas, asignaturas y grupos, separado del de estudiantes.
- Guion de pruebas completo, incluidos los errores controlados.

**Parte 2**

- Paquete `common` con API, excepciones y manejador global.
- Módulos `estudiantes`, `programas`, `asignaturas`, `grupos`, `matriculas`.
- Ports `EstudianteQueryPort` y `GrupoQueryPort` en sus módulos dueños.
- `matriculas` depende de los Ports y de ningún repositorio ajeno, verificado por script.
- No regresión: el guion produce una salida idéntica.

## 5. Conclusión

Lo que más me sirvió de este laboratorio fue ver que el cambio de la Parte 2 es **pequeño en código y grande en consecuencias**. Sustituir dos repositorios por dos interfaces de un método no alteró ni una línea de las reglas de negocio, y sin embargo cambió quién puede romper a quién: antes, modificar la persistencia de estudiantes obligaba a tocar matrículas; ahora matrículas depende de un contrato que puede implementarse de cualquier manera.

Eso conecta directamente con la conclusión del informe comparativo de arquitecturas: la opción hexagonal con módulos se justifica no porque resuelva un problema de escala que este sistema no tiene, sino porque desacopla a costo bajo. Y hay un detalle que lo confirma: si algún día el módulo de estudiantes se extrajera a un servicio aparte, bastaría escribir un adaptador que implemente `EstudianteQueryPort` llamando por HTTP, sin tocar la lógica de matrícula. La preparación para ese cambio ya está hecha, y no costó casi nada.

Los dos hallazgos de proceso me parecen igual de útiles que el resultado. El primero es la carrera del guion con los seeders, que me recordó que aceptar conexiones y estar listo son cosas distintas, el mismo malentendido que vimos con `depends_on` en Docker Compose. El segundo es que una regla arquitectónica sin verificación automática no sobrevive: `verificar-arquitectura.sh` ocupa cuarenta líneas y es lo único que impide que el acoplamiento regrese en silencio dentro de seis meses.